

Tarefa 9: Regiões Críticas Nomeadas vs Locks Explícitos

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Objetivo

Esta tarefa tem como objetivo implementar inserções paralelas em listas encadeadas utilizando `#pragma omp task`, garantindo a integridade dos dados sem condições de corrida. São abordados dois mecanismos de sincronização distintos:

- Regiões críticas nomeadas (`#pragma omp critical(nome)`) para um número fixo de listas, demonstrando paralelismo entre listas distintas;
- Locks explícitos (`omp_lock_t`) para um número M de listas definido em tempo de execução, onde o uso de `critical` nomeado torna-se estruturalmente impossível.

2. Ambiente de Execução

Item	Valor
CPU	AMD Ryzen 5 5600G
Núcleos físicos	4
Compilador	GCC com <code>-O2 -fopenmp</code>
Número de inserções	1000
Threads	4

3. Implementação

3.1 Estrutura da Lista Encadeada

A lista encadeada utiliza inserção no início ($O(1)$), com controle de sincronização externo ao nó:

```
typedef struct no { int val; struct no *next; } no_t;
```

```
static void inserir(no_t **head, int val) {  
    no_t *n = malloc(sizeof(no_t));  
    n->val = val; n->next = *head; *head = n;  
}
```

3.2 Parte 1 — 2 Listas com `#pragma omp critical` Nomeado

Quando o número de listas é fixo (neste caso, 2), é possível nomear cada região crítica individualmente. Os nomes `critical(lista0)` e `critical(lista1)` correspondem a mutexes distintos no runtime do OpenMP: uma thread inserindo na lista 0 não bloqueia outra inserindo na lista 1.

```
#pragma omp parallel num_threads(4)
#pragma omp single
{
    for (int i = 0; i < N; i++) {
        #pragma omp task firstprivate(i)
        {
            unsigned seed = (unsigned)i * 2654435761u;
            int escolha = rand_r(&seed) % 2;
            if (escolha == 0) {
                #pragma omp critical(lista0)    /* mutex A */
                inserir(&lista0, i);
            } else {
                #pragma omp critical(lista1)    /* mutex B */
                inserir(&lista1, i);
            }
        }
    }
}
```

3.3 Parte 2 — M Listas com omp_lock_t Explícito

Quando M é definido em tempo de execução (via argumento de linha de comando), não é possível utilizar critical nomeado — o nome precisa ser um identificador C fixo no código-fonte. A solução é alocar um vetor de omp_lock_t com um lock por lista:

```
omp_lock_t *locks = malloc(M * sizeof(omp_lock_t));
for (int i = 0; i < M; i++) omp_init_lock(&locks[i]);
```

```
#pragma omp parallel num_threads(4)
#pragma omp single
{
    for (int i = 0; i < N; i++) {
        #pragma omp task firstprivate(i)
        {
            unsigned seed = (unsigned)i * 2654435761u;
            int escolha = rand_r(&seed) % M;
            omp_set_lock(&locks[escolha]);
            inserir(&listas[escolha], i);
            omp_unset_lock(&locks[escolha]);
        }
    }
}
```

```
for (int i = 0; i < M; i++) omp_destroy_lock(&locks[i]);
```

4. Resultados

4.1 Parte 1 — 2 Listas, critical Nomeado

Lista	Elementos	Status
Lista 0	500	✓
Lista 1	500	✓

Total	1000	Correto
-------	------	---------

As 1000 inserções foram distribuídas uniformemente entre as duas listas (≈50% cada). Nenhuma condição de corrida foi detectada — o total permanece exato.

4.2 Parte 2 — 5 Listas, omp_lock_t (./critical 5)

Lista	Elementos	Status
Lista 0	189	✓
Lista 1	206	✓
Lista 2	208	✓
Lista 3	203	✓
Lista 4	194	✓
Total	1000	Correto

A distribuição entre as 5 listas é aproximadamente uniforme (esperado: 200 elementos por lista). A variação observada (189–208) decorre do gerador pseudoaleatório por semente, não de erros de sincronização. O total permanece 1000.

Com 8 listas (./critical 8), cada lista recebe aproximadamente 125 elementos, mantendo o total correto.

5. Análise

5.1 Por que critical Nomeado Não é Suficiente para M Dinâmico

O nome em `#pragma omp critical(nome)` deve ser um identificador C válido, fixo no código-fonte em tempo de compilação. Não é possível usar variável ou expressão como nome:

```
/* ERRO DE SINTAXE — o compilador rejeita */
#pragma omp critical(locks[i])
inserir(&listas[i], val);
```

Para $M = 5$, seria necessário escrever manualmente cinco blocos com nomes diferentes (lista0 ... lista4). Para M arbitrário definido em argv, isso é estruturalmente impossível sem alterar o código-fonte.

5.2 Semântica dos Mecanismos de Sincronização

Cada elemento `locks[i]` é um mutex independente — semanticamente equivalente a um critical nomeado único, mas criado e selecionado em runtime. Isso permite M listas com M locks onde M é qualquer valor definido pelo usuário.

Recurso	Nome/Índice	Criação	M Dinâmico
<code>#pragma omp critical(nome)</code>	Identificador fixo	Compilação	Impossível
<code>omp_lock_t</code>	Variável/Índice	Runtime	Sim

O paralelismo entre listas distintas é preservado em ambas as abordagens: apenas threads que disputam a mesma lista (mesmo índice) se bloqueiam mutuamente.

5.3 Ciclo de Vida do `omp_lock_t`

O gerenciamento correto do ciclo de vida dos locks é essencial para evitar vazamentos de recursos e comportamento indefinido:

- `omp_init_lock(&locks[i])` — inicializa o lock antes do uso paralelo;
- `omp_set_lock` / `omp_unset_lock` — adquire e libera o mutex em cada tarefa;
- `omp_destroy_lock(&locks[i])` — libera recursos após a região paralela.

6. Conclusão

A tarefa demonstrou dois mecanismos complementares de sincronização no OpenMP para proteção de estruturas de dados compartilhadas:

- Regiões críticas nomeadas são eficientes e expressivas quando o número de recursos a proteger é fixo e conhecido em tempo de compilação, permitindo paralelismo entre seções com nomes distintos;
- Locks explícitos (`omp_lock_t`) são indispensáveis quando a quantidade de recursos é definida em tempo de execução, pois permitem alocação dinâmica e indexação por variáveis.

Ambas as abordagens garantiram integridade total das listas encadeadas: o total de inserções foi sempre 1000, sem perdas ou duplicações causadas por condições de corrida. A escolha entre os mecanismos deve ser guiada pelo grau de dinamismo do programa — `critical` nomeado para casos estáticos, `omp_lock_t` para casos parametrizáveis.

7. Código Fonte

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define N      1000
6  #define NTHREADS 4
7  #define M_PADRAO 5
8
9  /* — Lista encadeada ————— */
10 typedef struct no { int val; struct no *next; } no_t;
11
12 static void inserir(no_t **head, int val) {
13     no_t *n = malloc(sizeof(no_t));
14     n->val = val;
15     n->next = *head;
16     *head = n;
17 }
18
19 static int tamanho(const no_t *h) {
20     int c = 0;
21     while (h) { c++; h = h->next; }
22     return c;
23 }
24
25 static void liberar(no_t *h) {
26     while (h) { no_t *t = h->next; free(h); h = t; }
27 }
28
29 /* — Parte 1: 2 listas — #pragma omp critical nomeado ————— */
30 static void partel(void) {
31     no_t *lista0 = NULL, *lista1 = NULL;
32
33     #pragma omp parallel num_threads(NTHREADS)
34     #pragma omp single
35     {
36         for (int i = 0; i < N; i++) {
37             #pragma omp task firstprivate(i)
38             {
39                 unsigned seed = (unsigned)i * 2654435761u;
40                 int escolha = (int)(rand_r(&seed) % 2);
41
42                 if (escolha == 0) {
43                     #pragma omp critical(lista0)
44                     inserir(&lista0, i);
45                 } else {
46                     #pragma omp critical(lista1)
47                     inserir(&lista1, i);
48                 }
49             }
50         }
51     }
```

```

52
53     int t0 = tamanho(lista0), t1 = tamanho(lista1);
54     printf("=== Parte 1: 2 listas, critical nomeado (%d threads) ===\n", NTHREADS);
55     printf("  Lista 0: %d elementos\n", t0);
56     printf("  Lista 1: %d elementos\n", t1);
57     printf("  Total  : %d %s\n\n", t0 + t1,
58           t0 + t1 == N ? "(correto)" : "(ERRO)");
59
60     liberar(lista0);
61     liberar(lista1);
62 }
63
64 /* — Parte 2: M listas - omp_lock_t explícito ————— */
65 static void parte2(int M) {
66     no_t **listas = calloc(M, sizeof(no_t *));
67     omp_lock_t *locks = malloc(M * sizeof(omp_lock_t));
68     for (int i = 0; i < M; i++) omp_init_lock(&locks[i]);
69
70     #pragma omp parallel num_threads(NTHREADS)
71     #pragma omp single
72     {
73         for (int i = 0; i < N; i++) {
74             #pragma omp task firstprivate(i)
75             {
76                 unsigned seed = (unsigned)i * 2654435761u;
77                 int escolha = (int)(rand_r(&seed) % M);
78
79                 omp_set_lock(&locks[escolha]);
80                 inserir(&listas[escolha], i);
81                 omp_unset_lock(&locks[escolha]);
82             }
83         }
84     }
85
86     printf("=== Parte 2: %d listas, omp_lock_t (%d threads) ===\n", M, NTHREADS);
87     int total = 0;
88     for (int i = 0; i < M; i++) {
89         int t = tamanho(listas[i]);
90         printf("  Lista %d: %d elementos\n", i, t);
91         total += t;
92         liberar(listas[i]);
93     }
94     printf("  Total  : %d %s\n", total,
95           total == N ? "(correto)" : "(ERRO)");
96
97     for (int i = 0; i < M; i++) omp_destroy_lock(&locks[i]);
98     free(locks);
99     free(listas);
100 }
101
102 int main(int argc, char *argv[]) {
103     int M = (argc > 1) ? atoi(argv[1]) : M_PADRAO;
104     if (M < 2) { fprintf(stderr, "M deve ser >= 2\n"); return 1; }
105
106     parte1();
107     parte2(M);
108     return 0;
109 }
110

```